E-commerce Assignment Help

Share

Show more

Matching the Sale Report's columns to the target schema

I'd mix, but keep the real part to one dataset. I'm going from memory of these datasets' columns, so check them against the actual files.

### Why the Kaggle "Sale Report.csv" suits your schema

Your tables look closely modelled on it. It has Order ID, Date, Status, Fulfilment, Style, SKU, ASIN, Courier Status, Qty, Currency, Amount, ship city/state/postcode/country, and a B2B flag. That maps almost directly onto ORDERS, ORDER_ITEM, PRODUCT, SHIPMENT, ADDRESS and `Is_Business`. Loading it is mostly column renaming and de-duplication, which is less work than inventing consistent values for those fields.

### What it lacks, so you generate it

- CUSTOMER (names, emails, phone, registered date, customer IDs), since the dataset has no customer identity.
- PAYMENT, which it doesn't record.
- `Stock_Quantity`, and `Product_Name` if missing.
- CATEGORY, if the file doesn't give a usable one.

### Why I wouldn't mix several real datasets

The other two (the Pakistan one and the UK `data.csv`) use different currencies, ID schemes and product catalogues, and have no shared keys with the first. Merging them creates cleaning work with little benefit. You'd be reconciling SKUs and categories across sources that were never meant to join.

### Why not generate everything

It is the least effort to code (a Faker script), but you'd have to invent realistic distributions for prices, order sizes, statuses and cities yourself. Real data also gives you messy edge cases that make your queries more convincing. If you only need a few thousand rows to test queries, full generation is perfectly defensible, and I'm not claiming real data is required.

### Pitfalls to plan for

1. Orders have no customer in the source. Generate customers first, then assign each order to one at random, so some customers have several orders.
2. `ADDRESS.ID` is a foreign key to the customer, so create address rows from the order's ship city/state/postcode after assigning the customer, and point `ORDERS.AddressID` at them.
3. In the source, `Amount` is a line total, so derive `Unit_Price` as Amount divided by Qty, and use one consistent value per SKU (e.g. the median). Cancelled rows often have Amount 0.
4. Order IDs repeat across lines in the source, which is fine for `ORDER_ITEM`, but you must generate `Line_Number` per order and keep `ORDERS` unique.
5. `SHIPMENT` has a UNIQUE `Order_Id`, so only one shipment per order.
6. Keep PAYMENT consistent with order status (for example, cancelled orders get Declined).

If you tell me which dataset you've downloaded, I can write the Python or SQL load script, starting with the Sale Report.

> it says i just need enough data so a few thousand rows should be enough so generating the whole should the trick no? especially if i map out the edge cases so that the data seems more random and realistic

Weighing synthetic data against real edge cases ▸

Yes, generating everything is the better choice for your situation. When I said real data was easier, I was assuming you needed volume or wanted realistic messiness for free. With a few thousand rows and a stated allowance to generate, the trade-off flips:

- You control referential integrity from the start. Every FK is valid by construction, with no cleaning, de-duplication or key reconciliation.
- One script, one seed, fully reproducible. Real data would need a cleaning step, a generation step and a join step.

- You can guarantee the cases your queries need to exercise, rather than hoping the dataset contains them.

The one risk is that generated data can look suspiciously uniform. Your plan of deliberately mapping out edge cases is the right fix, but "random" alone won't do it. You need skewed distributions:

- **Customers per order:** a long tail, so a few customers have many orders and most have one or two (a Zipf or geometric draw, not uniform).
- **Products:** a few best sellers, many rarely ordered, and some never ordered (useful for LEFT JOIN and anti-join queries).
- **Order status and payment:** mostly delivered and accepted, with some cancelled, returned, pending and declined. Keep these consistent, e.g. a cancelled order has a Declined or no payment, and a declined payment never has a delivered shipment.
- **Time:** order dates after the customer's `Registered_Date`, with seasonal peaks rather than a flat spread.
- **NULLs and oddities:** customers with no email or phone, addresses with no postcode or state, customers with no orders, a customer with several addresses, orders with many lines, a stock quantity of 0, and `Is_Business` customers with larger baskets.
- **Money:** `Amount` in `ORDER_ITEM` should equal quantity times unit price (or reflect a discount you apply deliberately), and `PAYMENT.Amount` should equal the order total.

Two caveats. First, check whether your assignment or marking criteria reward using real data. You said it's allowed, not required, but it's worth confirming. Second, "a few thousand rows" is the total or per table? A sensible shape would be around 300 customers, 500 addresses, 50 products, 1,500 orders and 4,000 order lines, but I'd rather you confirm the target than have me assume it.

If you want, tell me your DBMS (PostgreSQL, MySQL, SQLite or other) and the row counts, and I'll write a seeded Python script with Faker that outputs CSVs or INSERT statements in FK-safe order, with the edge cases above built in.

ST207 Databases — Assignment 1 (AT2026) Question 1 — Conceptual/E-R modelling Assessed: Suitability and completeness of the proposed e-commerc…

PASTED

does it say it rewards using real data in the making criteria here: "

No, it doesn't reward real data as such. Nothing in the criteria requires it, and it is never mentioned as a scoring factor. What Question 4 rewards is the quality of the data, however you obtain it.

The wording that matters:

- **Merit:** the data is "sufficiently varied and consistent to demonstrate all proposed use cases" and is "adequately documented, with data sources or generation procedures explained". The phrase "data sources or generation procedures" treats both routes as equally valid.
- **Distinction:** the data is "thoughtfully designed or selected to approximate realistic business conditions". "Designed or selected" again puts generated and real data on equal footing. It also asks for "relevant edge cases, varied customer or product behaviour, and less common but important scenarios", plus "data generation, processing and consistency checks … clearly documented".
- **Fail:** insufficient or inconsistent data, unpopulated relationships, or a "data generation process" that is "missing or inadequately explained".

So generating everything is safe, and arguably the easier route to a Distinction, because you can build the edge cases in deliberately. Real data would earn nothing extra, and it would add a "processing" step you'd have to document.

The criteria do tell you what to put in your report:

1. **Document the generation procedure.** Explain the script, the seed, the distributions you chose and why they approximate real business behaviour.
2. **Include consistency checks.** Run queries that prove integrity, such as no orders before the customer's registration date, payment totals matching order totals, and cancelled orders having no accepted payment. Show their output.
3. **List the edge cases explicitly.** Customers with no orders, products never ordered, NULL emails, multi-address customers, declined payments and so on.
4. **Tie the data to your use cases.** Q4 is judged on whether the data can "demonstrate the proposed use cases", so choose your Q5 queries first or

alongside the generator. If a query needs basket analysis or trends, the data must contain real co-purchase patterns and time variation, or the results will look flat.

5. **Plan for Q6.** Your trigger demos need both a valid operation and a violating one, so keep a few rows or scenarios set aside for those tests.

One caveat: I'm reading only the rubric you pasted. If the assignment brief or a Moodle announcement states a size requirement or a data restriction, that would override this, so skim it once for anything about row counts or sources.

> so i can generate the names, emails, ... using faker and the rest just normally with a list or a dictionary of things and giving these things different weights and adding special cases on top of that to make the edge cases to make more realistic. that should work right?

Mapping dependencies between generated columns ▸

Yes, that approach works. The main risk is making every column an independent weighted random pick, which produces data that is individually plausible but internally incoherent. Generate in dependency order and derive values rather than drawing them.

### Where independent draws go wrong

- **Product attributes:** draw price, category, style and size once per product, then store them. Don't redraw them per order line.
- **Order lines:** `Amount` should be computed as quantity times the product's `Unit_Price` (minus any discount you apply deliberately), never drawn separately.
- **Payments and shipments:** these should be derived from the order's status, not sampled on their own. A cancelled order should not end up with an accepted payment and a delivered shipment.
- **Addresses:** city, postcode, state and country must agree. Faker's locale providers (`en_GB`, `fr_FR`, `de_DE`) give you coherent combinations, whereas mixing a random city list with random postcodes does not.
- **Dates:** generate registration date first, then order date after it, then payment date on or after the order date.

### Suggested generation order (FK-safe)

1. CATEGORY, then PRODUCT
2. CUSTOMER, then ADDRESS

3. ORDERS, then ORDER_ITEM
4. PAYMENT and SHIPMENT, derived from each order's status and total

### Faker-specific traps for your schema

- `Email` is `UNIQUE`, so use `fake.unique.email()` or build emails from an index, or the load will fail midway.
- `Phone` is `VARCHAR(20)` and Faker phone numbers sometimes include extensions that exceed it, so truncate or pick a format.
- `MInitial` is one character, and `FName` / `LName` have length limits, so check lengths.
- Seed both `random.seed(n)` and `Faker.seed(n)`, otherwise your output is not reproducible, which the Presentation criteria care about.

### On weights

Hand-picked weights are fine for categorical things like status and payment method. For product popularity and orders per customer, use a skewed distribution (Zipf, or `random.choices` with weights like `1/rank`) rather than inventing numbers by hand. For basket analysis, also define a few co-purchase affinities, for example a product group that tends to be bought together, otherwise your Q5 results will show no pattern.

### Edge cases: inject, don't hope

Random weights give rare cases only by chance, and with a few thousand rows a 1% case might appear zero times. Add the special cases deterministically after the bulk generation: for example, exactly 10 customers with no orders, 5 products never ordered, 20 customers with NULL phone, 15 with several addresses, and a few orders with 10+ lines. Then your report can state exactly what is covered and why.

### Before loading

Write SQL checks for the invariants (order date after registration, line totals matching, payment total equal to order total, status consistency) and run them on the loaded data. Their output goes straight into the Q4 documentation.

Which DBMS are you using? I can then write the generator to output CSVs or INSERT statements in the right order.

so for example if i put function that says for i in range... if i%20 == 0 then is_business = 1. would that be better than for example using a random function with the same logic for i in range:... if randint(1,20) % 20 == 0 then is_business?

Comparing exact quotas with random variance across sample sizes ▸

Neither is ideal. The best option is a third one: pick an exact number of customers at random.

### Why `i % 20 == 0` is weaker than it looks

- It gives exactly 5%, but it makes the pattern visible. Every business customer has an ID divisible by 20, and anyone who sorts by ID will spot it.
- It confounds with any other modulo rule. If you also use `i % 4` or `i % 5` for something else (status, country, whatever), every business customer will land in the same bucket for that attribute, because 20 is divisible by both. Your queries would then show a fake correlation, such as "all business customers are in Germany".
- If IDs are assigned in registration order, business customers are evenly spaced through time, which looks artificial in any trend query.

### Why `randint(1, 20) % 20 == 0` is also weaker

It is just a 5% coin flip (the `% 20` is redundant, as it is the same as `randint(1, 20) == 20`). You get no pattern, but you don't control the count. With 300 customers you expect 15 business customers, but the standard deviation is about 3.8, so 8 or 22 would not be surprising. If a query needs business customers to exist, or you want to state exact figures in your report, that uncertainty is a nuisance. With a seed it is reproducible, but you cannot know the count in advance.

### What I'd do: sample an exact set

python

```
import random
random.seed(42)

n_customers = 300
business_ids = set(random.sample(range(1, n_customers + 1), k=15))  # exact

for i in range(1, n_customers + 1):
    is_business = 1 if i in business_ids else 0
```

This gives you an exact, documentable count (15 business customers) with no positional pattern, and it stays reproducible under the seed.

**Rule of thumb**

- **Edge cases and guaranteed counts** (customers with no orders, products never ordered, NULL phones): use exact sampling like above.
- **Background behaviour** (which product a line item picks, which payment method, order date): use random draws with weights, since small variation there is realistic.
- **Avoid `i % k` for anything meant to look random.** It is fine for things like assigning IDs or cycling through categories to guarantee coverage, but not for attributes that should be independent of each other.

One more point: if you want business customers to behave differently (larger baskets, more orders), assign the flag first and let it drive the later draws, rather than generating orders first and flagging afterwards.

how is that different to `randint(1, 20) == 20` explain shortly and in simple terms

`randint(1, 20) == 20` gives each customer their own separate 1-in-20 chance. Each customer is decided independently, so the total number of business customers varies. You might get 9 one time and 21 another.

`random.sample(range(1, 301), k=15)` picks exactly 15 customers in one go, like drawing 15 names from a hat. The total is always 15, but which customers are picked is still random.

So the difference is in the count, not in how random it looks:

- `randint` **per customer:** about 5% on average, but the exact number is unpredictable.
- `random.sample`: exactly 5%, every time.

This matters because you want to write "15 business customers" in your report and know your queries will always have business customers to find. With a fixed seed, `randint` would also give the same result every run, but you would only find out the count after generating the data.

> so you could then choose say to pick a number of names from the hat which makes the number of names picked out compared to the total number of names seem random

Weighing whether randomising the count is worthwhile ▸

Yes. You pick the number first, then draw that many names from the hat:

```python
k = random.randint(10, 20)  # somewhere between about 3% and 7% of 300
business_ids = set(random.sample(range(1, n_customers + 1), k=k))
```

The proportion now varies, but only within limits you set. Unlike a per-customer coin flip, it can't wander to 2 or 40, and it can never be zero, so your queries will always find business customers.

With a fixed seed, `k` is the same every run, so it stays reproducible. You just print the final count and quote it in your report ("generated 14 business customers").

I'd only do this if you want the ratio to look less tidy. Exactly 5% is not unrealistic, and the markers won't care. For edge cases you need guaranteed (customers with no orders, say), keep a fixed `k` so the count is known in advance.

Show more

Settling on removing the inner loop ▸

The main bug is the second loop, `for i in range(1, N_CUSTOMERS + 1)`, nested inside the first. Everything after it (weights, quiet customers, date, `append`) runs inside the inner loop, so each customer is appended N times. With 300 customers you get 90,000 rows, and `cid` stays fixed during the inner loop, so you get repeated IDs and a primary key clash on load.

You only need one loop, and you can use `cid` for the business check. Here is the corrected version:

python

```
from datetime import date, timedelta

customer_rows = []
customer_weights = []
quiet_customers = []

k = random.randint(6, 12)
business_ids = set(random.sample(range(1, N_CUSTOMERS + 1), k=k))

for cid in range(1, N_CUSTOMERS + 1):
    fname = fake.first_name()
    lname = fake.last_name()
    email = fake.unique.email()
    phone = fake.phone_number()[:20]          # VARCHAR(20)

    initial = None if random.random() < 0.6 else random.choice("ABCDEFGHIJK
    is_business = 1 if cid in business_ids else 0

    roll = random.random()
    if roll < 0.10:
        customer_weights.append(10)
    elif roll < 0.40:
        customer_weights.append(3)
    else:
        customer_weights.append(1)

    if random.random() < 0.15:
        quiet_customers.append(cid)

    registered = date(2023, 1, 1) + timedelta(days=random.randint(0, 729))

    customer_rows.append((initial, lname, fname, cid, email, phone,
                          registered.isoformat(), is_business))
```

Two other points:

1. **Indentation:** as pasted, nothing under the `for` lines is indented, which would be an `IndentationError`. This may just be a paste artefact, but check that the loop body is indented.
2. **Customers can be both quiet and heavy:** `customer_weights` and

`quiet_customers` are drawn independently, so a customer can have weight 10 and also be in the quiet list. When you generate orders, either exclude quiet customers from the weighted draw or decide which wins, otherwise your "customers with no orders" edge case won't hold.

A quick sanity check after the loop: `assert len(customer_rows) == N_CUSTOMERS == len(customer_weights)`. It would have caught this bug immediately.

Show more

Mapping the schema's constraints to each rubric point ▸

Question 2 is the **relational model** section of the report (the third bullet). It is the step after the E-R diagram: you turn the diagram into tables. The 1.5 pages are not for repeating the table definitions, since the DDL is submitted separately. They are for explaining **what changed between the E-R model and the tables, and why**.

Here is what each part asks for, using your own schema.

### (i) Adaptations from the E-R model

Say what the diagram had that cannot be a table directly, and how you handled it. Likely examples:

- `ORDER_ITEM` resolves the many-to-many relationship between ORDERS and PRODUCT. `Quantity`, `Amount`, `Currency` and `Line_Number` are relationship attributes, and the composite primary key is `(Line_Number, Order_Id)`.
- A composite attribute such as customer name is flattened into `FName`, `MInitial` and `LName`.
- The one-to-one relationship between ORDERS and SHIPMENT is implemented as a foreign key with `UNIQUE (Order_Id)`.
- `ADDRESS` is its own table with a foreign key to CUSTOMER, because a customer

can have several addresses.

### (ii) Views

State which views you made, which columns they contain and which base tables they come from. Your pasted schema has no views. They are optional, and the rubric only rewards them if the use case justifies them, for example an `ORDER_TOTALS` view. If you have none, say so in one sentence.

### (iii) Constraints for minimum and maximum cardinalities

Explain how “mandatory” and “optional”, and “one” and “many”, appear in the tables:

- **Mandatory participation:** `NOT NULL` on a foreign key. Every order must have a customer and an address.
- **Optional participation:** the absence of such a requirement. A customer can have zero orders, and an order can have no shipment yet.
- **Maximum of one:** `UNIQUE`. At most one shipment per order.
- **Limits of SQL:** “every order has at least one item” cannot be enforced by declarative constraints, because the order must exist before its items do. Say that this is handled by a trigger or by the loading procedure. Markers like seeing that you know where the limits are.

### (iv) Anything else

This covers choices such as:

- Storing `Amount` as a line total, so historical prices stay correct if `Unit_Price` changes later.
- Not storing the order total, because it is derived.
- `Is_Business` as 0/1 with a `CHECK`.
- `Payment_Status` restricted by a `CHECK`.
- Nullable `Email` and `Phone`, with `Email` still `UNIQUE`.

Two weaknesses in your schema are worth addressing in the report, or fixing:

1. Nothing stops an order from using an address that belongs to a different customer, because `ORDERS.AddressID` only references ADDRESS. A composite foreign key would fix this, or you could use it as a Q6 trigger.
2. `Currency` sits on each order line, while PAYMENT has no currency. Either justify this or make it consistent.

Also note the data bullet says you must submit **chat logs** if you used an AI tool. Export or save this conversation now so you don’t have to reconstruct it later.

so i don't have to explain each metrics data type and why i have chosen that

Deciding which data types need explaining ▸

Correct, you don't need to go through every attribute. The brief lists four things for that section (adaptations, views, cardinality constraints, other adaptations), and a per-column table of data types is not one of them. It would also eat your 1.5 pages quickly.

The rubric does mention "appropriate data types" under Q2, but markers judge that from your DDL. The report only needs to explain choices where a reader might ask "why?". A few lines under (iv) is enough, for example:

- **Money as `NUMERIC(10,2)`:** exact decimal arithmetic, whereas `FLOAT` gives rounding errors in totals.
- `Order_Id` **as** `VARCHAR`**:** if it is a formatted identifier rather than a counter, say so, since the other IDs are `INT`.
- `Is_Business` **as** `INT` **with a** `CHECK (0, 1)`**:** stands in for a boolean, which not every DBMS supports.
- `Payment_Date` **or** `Order_Date` **as** `DATE`**, not a timestamp:** only if you can justify that daily granularity is enough for your use cases.

Skip the obvious ones ( `FName` as `VARCHAR(40)`, `ID` as `INT`) and spend the space on the points above plus cardinality constraints and the E-R adaptations.

what are the methods i can use to make sure that my database is consistent, through DB Brwoser

Reviewing SQLite foreign key and integrity settings ▸

I'm assuming DB Browser for SQLite. If you use another DBMS, tell me, because some of this differs.

### 1. Built-in checks (declarative)

- **Foreign keys:** SQLite does not enforce them unless `PRAGMA foreign_keys = ON;` is set per connection. DB Browser usually has this enabled under Preferences > Database > Foreign keys, but check it, and put the PRAGMA at the

top of your script too.

- **Find existing violations:** `PRAGMA foreign_key_check;` returns one row per broken foreign key (empty means clean). This is the quickest check after a bulk load.
- **Database health:** `PRAGMA integrity_check;` should return `ok`. It checks file and index corruption, not your business rules.
- **PRIMARY KEY, UNIQUE, NOT NULL and CHECK** reject bad rows at insert time. If a load fails with a constraint error, that is the check doing its job.

### 2. Your own consistency queries (the important part)

Write queries that should return zero rows, and screenshot them for the report. For your schema:

sql

```
-- Orders placed before the customer registered
SELECT o.Order_Id FROM ORDERS o
JOIN CUSTOMER c ON c.ID = o.ID
WHERE o.Order_Date < c.Registered_Date;

-- Order uses an address belonging to another customer
SELECT o.Order_Id FROM ORDERS o
JOIN ADDRESS a ON a.AddressID = o.AddressID
WHERE a.ID <> o.ID;

-- Payment total differs from order total (accepted payments)
SELECT p.Order_Id, p.Amount, SUM(oi.Amount) AS total
FROM PAYMENT p JOIN ORDER_ITEM oi ON oi.Order_Id = p.Order_Id
WHERE p.Payment_Status = 'Accepted'
GROUP BY p.Payment_Id HAVING p.Amount <> SUM(oi.Amount);

-- Orders with no items
SELECT o.Order_Id FROM ORDERS o
LEFT JOIN ORDER_ITEM oi ON oi.Order_Id = o.Order_Id
WHERE oi.Order_Id IS NULL;

-- Payment dated before the order
SELECT p.Payment_Id FROM PAYMENT p
JOIN ORDERS o ON o.Order_Id = p.Order_Id
WHERE p.Paid_At < o.Order_Date;
```

Add status checks too, such as cancelled orders with an Accepted payment or a delivered shipment.

### 3. Triggers (Q6)

Triggers cover rules that declarative constraints cannot express, for example:

- Reject an order whose `AddressID` belongs to a different customer.
- Reject an order line when `Quantity` exceeds `Stock_Quantity`, and decrement stock on a valid insert.

SQLite triggers reject with `SELECT RAISE(ABORT, 'message')`. Demonstrate one valid insert and one violating insert for each.

### 4. SQLite-specific traps

- **Types are loosely enforced:** `VARCHAR(40)` does not limit length, and a text value can sit in a numeric column (type affinity). Check with `SELECT typeof(col)` or `LENGTH(col)` if it matters, or use `CHECK (LENGTH(...) <= 40)`.
- **Dates are just text:** nothing stops `'2025-13-45'`. Use `WHERE date(Order_Date) IS NULL` to find invalid dates, and keep a single ISO format ( `YYYY-MM-DD` ).
- **Transactions:** wrap loads in `BEGIN; ... COMMIT;` so a failed load can be rolled back instead of leaving half-loaded tables. In DB Browser, use Write Changes or Revert Changes.

### 5. For the report

Keep the zero-row query outputs and the `PRAGMA foreign_key_check` result as evidence in the data section. The Q4 rubric asks for “consistency checks clearly documented”, and this is exactly that.

Show more

Checking the page budget for the screenshots

Here is a structure that fits the brief. Each use case or trigger is one block: heading, up to 2 paragraphs, then screenshots.

**Block pattern (repeat for each of the 5 blocks)**

1. **Heading with the question number**, e.g. "Q5, Use case 1: [short title]".
2. **Rationale, up to 2 paragraphs, covering the whole use case.** Do not write one per query. Paragraph 1: the business question and why it matters. Paragraph 2: how the queries work together and what the results show.
3. **Screenshots, one per query**, each with a one-line caption: "Query 1a: [what it returns]". If a result is long, crop to the first rows and say "first 10 of N rows" in the caption.

**Q5 (3 use cases)**

| Block | Rationale | Screenshots |
|---|---|---|
| Q5, Use case 1 | up to 2 paragraphs | 3 (queries 1a, 1b, 1c) |
| Q5, Use case 2 | up to 2 paragraphs | 3 (queries 2a, 2b, 2c) |
| Q5, Use case 3 | up to 2 paragraphs | 2 (queries 3a, 3b) |

Put the screenshots straight after the rationale, in query order. If the queries build on each other (for example a CTE feeding a ranking), say so in paragraph 2 so the markers see the progression.

**Q6 (2 triggers)**

I'm assuming "three trigger codes" means three `CREATE TRIGGER` statements that together enforce one rule (for example BEFORE INSERT, BEFORE UPDATE and an AFTER trigger). Tell me if it means something else.

Trigger 1 has 3 pieces of code and trigger 2 has 2, but the code goes in the separate file, so the report is about behaviour:

1. **Heading:** "Q6, Trigger 1: [rule it enforces]".
2. **Rationale, up to 2 paragraphs.** Paragraph 1: the integrity risk the schema cannot prevent on its own and why declarative constraints are not enough. Paragraph 2: what each of the three pieces does and why you needed all three.
3. **Screenshots:**
   - The relevant rows **before** the test, if state matters (e.g. stock level).
   - **Valid operation:** the statement and the successful result.
   - **Violating operation:** the statement and the error message (the `RAISE` text).
   - The rows **after** the violating test, showing nothing changed.

Repeat for Trigger 2 with its two pieces.

I would show the before and after states, not only the error. The rubric rewards evidence of "database state" and no unintended side effects, and an error message alone does not prove nothing was changed.

### Where to put everything

- Candidate number at the very top of page 1 only.
- Q5 and Q6 sections come last, after the data section.
- If you used Python anywhere (for example to produce a result), include that code and its output in the relevant block.
- Do not add a separate "all screenshots" appendix. Each result belongs directly under its rationale.

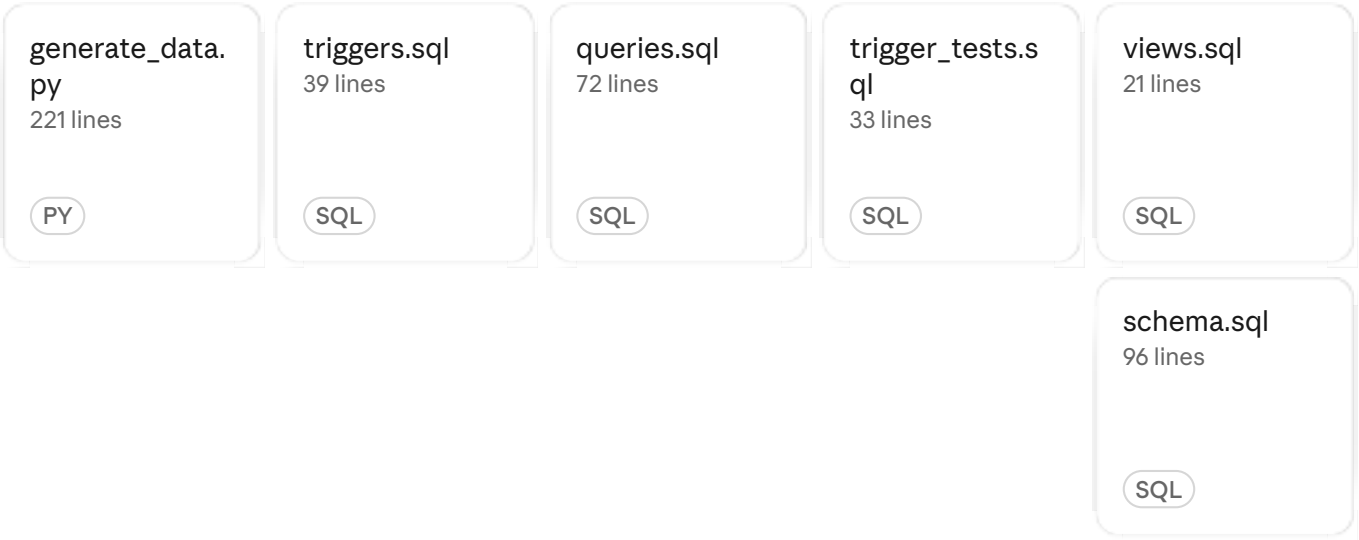
### A warning on length

The earlier sections take up to 5 pages (1.5 + 1.5 + 1 + 1), leaving about 3 pages for 5 blocks. You have at least 8 screenshots for Q5 and 4 to 8 for Q6. That is tight. To make it fit:

- Crop screenshots tightly to the result grid and put two side by side in a two-column layout.
- Keep each rationale to about 100 to 150 words. "Up to 2 paragraphs" is a maximum.
- Set the screenshot width so the text is still legible when printed. If it is not

legible, the brief warns of lost marks.
- If you still overflow, shorten the earlier sections first, since those are limits too (e.g. 1.5 pages), and the pages are better spent on results the markers may otherwise score zero for.

Check the page count in the final PDF, not the draft, because images often reflow.




here are the presentation requirements : "Presentation: ensure completeness and good quality of all outputs: legible, good resolution figures; diagrams created with professional tools (ERDPlus, draw.io, DBeaver etc); any data dictionary (for instance, a table describing each entity, attribute names, data types, constraints, description, sample values); and well-documented code (comments, query purpose and/or intended use). You must produce a complete and organised report, addressing all requested sections/information, along with a complete set of references and compliance with all necessary policies, including acknowledgement of generative AI usage and submission of chat logs. Check the Deliverables section for format and number of pages."
use that to make sure my files are presentable. but don't over do it, make simple, ONLY necessary and efficient comments please

Writing the files with consistent comments and British English spelling ▸

I'll tidy all six files with short purpose comments only. Code stays unchanged, apart from two bug fixes in `generate_data.py` and a setup block in the trigger tests, which I flag afterwards.

Created 6 files, shared files ▸

I've added short purpose comments to all six files. The only code changes are the two bug fixes in `generate_data.py` and the setup block in `trigger_tests.sql`. The first point below needs your decision.

**1. The files don't match what you described earlier.**

- `queries.sql` has 2 + 2 + 2 queries, not 3 + 3 + 2.
- `triggers.sql` has 2 + 2 pieces of code, not 3 + 2.
- The status query had no label, so I called it 2.2, assuming 2.1 is missing. I also named the use cases myself, so rename them to match your report.
- If these are older versions, send me the current ones and I'll redo the comments.

**2. `generate_data.py` had two bugs, which I fixed.**

- `random.sample` was given a float for `k`, which raises a `TypeError`. It now uses an integer count of 6 to 12% of customers.
- `is_business` tested `i` instead of `cid`. `i` was left over from the product loop, so only one customer could ever be a business. It now uses `cid`.
- Regenerating gives a different database from any earlier run. If you regenerate, retake the screenshots (customer 206 and the others). The script you submit must match the database you submit.

**3. `trigger_tests.sql` needed a setup block.**

- Product 9001 and order `ORD-TEST1` don't exist in the generated data, so test 1.1 would fail without them.
- I added a setup block with stock 10 and price 10.00, so test 1.1 takes stock to 7 and test 1.2 fails on 8 > 7.
- Change those values if you used different ones for your screenshots.

**4. Two data weaknesses the Q4 markers may notice.**

- Every order has exactly one line, so there is no multi-line basket. Worth fixing if you have time.
- Query 3.2 (customers who never ordered) relies on chance, not a guaranteed edge case. Check that it returns rows.

Remember to add your acknowledgement of generative AI use and the chat logs to the submission, since the presentation requirements ask for both.

Download all

Show more

Checking whether the chat log needs its own file

**ERD and relational schema: neither goes in the code file or the PDF report.** The brief says there's no need to include the model images in the report, because they should be submitted separately. Submit each as its own file in your GitHub repository:

- the ER model's source file (draw.io, ERDPlus or similar) and its exported image (PNG or PDF)
- the relational model's source file and exported image

Check that both images are legible when zoomed in. Neither belongs in the `.sql` or `.py` file.

**Chat log: also a separate file, not in the PDF.** The deliverables list asks for the chat log or source file used to generate the synthetic data, and the data section says to include chat logs as part of your deliverables. A pasted conversation would use up your 8-page limit, so keep it as its own file. Your `generate_data.py` counts as the source file. The PDF only needs the generative AI statement, plus a sentence in the data section saying the chat log is in the repository.

**How to export it.** As far as I know, there is no per-chat export button, so use one of these:

1. **Print to PDF (simplest):** open this conversation in a desktop browser, press Ctrl+P (Cmd+P on Mac), choose "Save as PDF", and scroll through afterwards to check nothing is cut off.
2. **Copy and paste:** select the whole conversation, paste it into a `.txt` or `.md` file and save it.
3. **Full account export:** Settings, then Privacy, then Export data. It emails you a zip of all your chats, but it is overkill and includes unrelated conversations.

Whichever you pick:

- **Redact identifying details:** the brief says to use only your candidate number, so remove any names or emails first.
- **Include every AI chat:** add any other conversation where you used an AI tool for this assignment, not just this one.
- **Leave the text unedited:** apart from the redactions, don't change it.

**Suggested repository contents:**

- ER model source and image
- relational model source and image
- `schema.sql`, `views.sql`, `queries.sql`, `triggers.sql`, `trigger_tests.sql`
- `generate_data.py`
- the database file, or a download link in the report if it is too large for GitHub
- the chat log
- the PDF report, with your candidate number at the top and the generative AI statement included

Your submission is due **today, 06/11/2026 at 3 pm**, on both GitHub and Moodle, so leave time for the Moodle step. I can't see the clock, so check how long you have left. Also check Moodle for the exact wording of the AI statement, since the brief says the School's policy applies.

> so i have a general folder which contains all of the files, give me the step by step process on everything i need to put inside relating to the chat logs, red model and relational schema

Planning redaction and naming for the chat-log files ▸

Here is a layout and checklist for the folder.

### Suggested folder structure

```
assignment1/
  report.pdf
  er_model/
    er_model.drawio      (source)
    er_model.png         (export)
  relational_model/
    relational_model.drawio   (source)
    relational_model.png      (export)
  sql/
    schema.sql
    views.sql
    queries.sql
    triggers.sql
    trigger_tests.sql
  data/
    generate_data.py
    E_Commerce.db        (or a download link in the report if too big)
  chat_logs/
    chat_log_1.pdf
```

Keep the folder names free of your own name. Use your candidate number if you want a prefix.

### 1. ER model

1. Make it in draw.io or ERDPlus, using Crow's foot notation.
2. Check it shows every entity from your schema, with attributes, primary keys and the cardinalities, both minimum (optional or mandatory) and maximum.
3. Save the source file (`.drawio`, or the ERDPlus export) into `er_model/`.
4. Export the image as PNG at a high scale (200% or more in draw.io: File, Export as, PNG) or as PDF. Zoom in to check the text is legible.
5. Put both the source and the image in `er_model/`.

### 2. Relational model

1. Draw it as its own diagram, separate from the ER model. It should show each table with its columns, primary keys marked, foreign keys marked, and arrows from each foreign key to the table it references.
2. Include the composite key of `ORDER_ITEM`, the `UNIQUE` on `SHIPMENT.Order_Id`, and the views if you want them shown.
3. Save the source and a high-resolution export into `relational_model/`.

It must match `schema.sql` exactly. Compare the two once for column names and types before exporting.

### 3. Chat logs

1. List every AI conversation you used for this assignment, not only this one.
2. Open each in a desktop browser, press Ctrl+P (Cmd+P on Mac), choose "Save as PDF", and save it to `chat_logs/` as `chat_log_1.pdf`, `chat_log_2.pdf` and so on.
3. Scroll through each PDF to check nothing is cut off.
4. Before saving, check for identifying information (name, email, student number) and remove it, since the brief says to use only your candidate number. If a conversation has too much to remove, copy it into a text file and redact it there instead.
5. Do not edit the rest of the content.

### 4. Before uploading

1. Delete anything not in the deliverables list, such as scratch files, old versions and `E_Commerce_empty.db`.
2. Check the PDF report mentions that the chat logs and `generate_data.py` are in the repository, and includes the AI statement and your candidate number at the top.
3. Make sure the `.db` file matches the current `generate_data.py` and your screenshots. If you regenerated the data after the fixes, the screenshots need retaking.

4. Upload everything to your `assignment1` repository on GitHub Classroom. If the `.db` is too large, upload it to Google Drive or Dropbox, set the link to accessible and put the link in the report.
5. Copy the repository URL, paste it into the Moodle portal and press **Submit**. The submission only counts once you do this.

The deadline is today, 06/11/2026 at 3 pm. Do the Moodle step first if time is tight, then tidy afterwards.